

Họ và tên : Trần Mạnh Hùng

MSSV: 23127195

## CÂU 1:

Hệ thống cần tích hợp nhiều dịch vụ bản đồ khác nhau để định vị GPS điểm đến:

- Google Maps
- VietBanDo
- Tương lai có thể thêm dịch vụ khác

Mỗi dịch vụ có API và định dạng trả về khác nhau.

### Yêu cầu:

1. Đề xuất một Design Pattern phù hợp để giải quyết bài toán này
2. Giải thích vì sao chọn mẫu đó.
3. Viết pseudo-code minh họa cách sử dụng.

### Giải

#### 1. Đề xuất Design Pattern: Adapter Pattern (Mẫu thiết kế Người thích nghi)

Trong trường hợp này, **Adapter Pattern** là lựa chọn tối ưu nhất. Ngoài ra, để hệ thống linh hoạt hơn trong việc chuyển đổi giữa các dịch vụ tại thời điểm chạy (runtime), có thể kết hợp nó với **Strategy Pattern**.

#### 2. Giải thích lý do lựa chọn

Việc sử dụng Adapter Pattern mang lại các lợi ích trực tiếp cho đồ án của này:

- **Thống nhất giao diện (Standardization):** Mỗi dịch vụ bản đồ (Adaptee) có cấu trúc dữ liệu và tên hàm khác nhau. Adapter giúp "bọc" các API này lại và cung cấp một giao diện chung duy nhất cho hệ thống của bạn sử dụng.
- **Tuân thủ nguyên tắc SOLID:**
  - **Single Responsibility Principle (SRP):** Mỗi lớp Adapter chỉ chịu trách nhiệm chuyển đổi cho một dịch vụ cụ thể.
  - **Open/Closed Principle (OCP):** Khi muốn thêm một dịch vụ bản đồ mới (ví dụ: Bing Maps), bạn chỉ cần tạo thêm một lớp Adapter mới mà không cần sửa đổi mã nguồn hiện tại của hệ thống.
- **Dễ bảo trì và kiểm thử:** Nếu API của Google Maps thay đổi, chỉ cần sửa mã trong GoogleMapsAdapter mà không làm ảnh hưởng đến logic điều phối của toàn hệ thống.

### 3. Pseudo-code minh họa (Dựa trên nền tảng Java)

Vì bạn đang quan tâm đến Java và các nguyên tắc thiết kế, dưới đây là cách triển khai mẫu:

#### Bước 1: Định nghĩa Interface chung cho hệ thống

```
interface IMapService {  
    // Trả về một đối tượng Coordinate chuẩn của hệ thống  
    Coordinate getGPSFromAddress(String address);  
}
```

#### Bước 2: Các lớp Adapter cho từng dịch vụ

// Adapter cho Google Maps

```
class GoogleMapsAdapter implements IMapService {  
    private GoogleMapsAPI googleAPI = new GoogleMapsAPI();  
  
    @Override  
    public Coordinate getGPSFromAddress(String address) {  
        // Giả sử Google trả về một chuỗi "lat,lng"  
        String result = googleAPI.searchLocation(address);  
        double lat = Double.parseDouble(result.split(",")[0]);  
        double lng = Double.parseDouble(result.split(",")[1]);  
        return new Coordinate(lat, lng);  
    }  
}
```

// Adapter cho VietBanDo

```
class VietBanDoAdapter implements IMapService {  
    private VietBanDoAPI vbdAPI = new VietBanDoAPI();  
  
    @Override  
    public Coordinate getGPSFromAddress(String address) {  
        // Giả sử VietBanDo trả về một Object phức tạp khác  
        VBDRestult vbdResult = vbdAPI.findPoint(address);  
        return new Coordinate(vbdResult.getLatitude(),  
vbdResult.getLongitude());  
    }  
}
```

```
}
```

### **Bước 3: Sử dụng trong Phân hệ Tổng đài (Nhân viên S2)**

```
class CallCenterService {  
    private IMapService mapService;  
  
    // Có thể thay đổi service dễ dàng qua Constructor hoặc Setter (Strategy  
    Pattern)  
    public CallCenterService(IMapService service) {  
        this.mapService = service;  
    }  
  
    public void processS2Step(String address) {  
        // Hệ thống không cần biết đang dùng Google hay VietBanDo  
        Coordinate gps = mapService.getGPSFromAddress(address);  
        System.out.println("Tọa độ xác định được: " + gps.getLat() + ", " +  
gps.getLng());  
        // Tiếp tục điều phối tài xế...  
    }  
}
```

## **Câu 2**

Hệ thống tính giá chuyển đi gồm:

- Giá cơ bản
- Phụ phí giờ cao điểm
- Phụ phí sân bay

- Mã giảm giá
- Phụ phí loại xe

Các rule có thể bật/tắt linh hoạt và có thể thay đổi thường xuyên.

### **Yêu cầu:**

1. Đề xuất một Design Pattern phù hợp.
2. Giải thích lợi ích so với cách dùng if-else.
3. Pseudo-code minh họa cách tính giá.

### **Giải**

#### **1. Đề xuất Design Pattern: Decorator Pattern (Mẫu thiết kế Trang trí)**

Để giải quyết bài toán tính giá với nhiều loại phụ phí và quy tắc có thể thay đổi, **Decorator Pattern** là lựa chọn phù hợp nhất. Mẫu thiết kế này cho phép thêm các trách nhiệm (ở đây là các khoản phụ phí hoặc giảm giá) vào một đối tượng một cách linh hoạt tại thời điểm chạy (runtime) mà không làm thay đổi cấu trúc của đối tượng đó.

#### **2. Giải thích lợi ích so với cách dùng cấu trúc if-else**

Việc sử dụng Decorator Pattern mang lại nhiều ưu điểm vượt trội so với việc sử dụng các câu lệnh điều kiện (if-else) lồng nhau:

- **Tránh "God Object" và mã nguồn cồng kềnh:** Nếu dùng if-else, hàm tính giá sẽ trở nên rất dài và phức tạp khi số lượng quy tắc (rule) tăng lên. Decorator chia nhỏ mỗi quy tắc thành một lớp (class) riêng biệt, giúp mã nguồn sạch và dễ đọc.
- **Tuân thủ nguyên tắc Open/Closed (OCP):** Khi cần thêm một loại phụ phí mới (ví dụ: phụ phí thời tiết xấu), chỉ cần tạo thêm một class Decorator mới mà không cần can thiệp hay sửa đổi mã nguồn hiện có của các lớp tính giá khác.
- **Kết hợp linh hoạt (Flexible Composition):** Cấu trúc if-else thường cố định thứ tự kiểm tra. Với Decorator, hệ thống có thể tùy ý kết hợp các loại phí (ví dụ: một chuyến đi vừa có phụ phí giờ cao điểm, vừa có

mã giảm giá) bằng cách "bao bọc" (wrap) các đối tượng theo thứ tự mong muốn.

- **Quản lý trạng thái bật/tắt dễ dàng:** Việc áp dụng hay loại bỏ một quy tắc tính giá chỉ đơn giản là việc thêm hoặc bớt lớp bao bọc bên ngoài đối tượng gốc, thay vì phải thay đổi logic điều kiện phức tạp bên trong hàm xử lý.

### 3. Pseudo-code minh họa (Java)

Dưới đây là cách triển khai hệ thống tính giá sử dụng Decorator Pattern:

#### Bước 1: Định nghĩa Interface chung cho việc tính giá

Java

```
interface ITripPricing {  
    double calculatePrice();  
}
```

#### Bước 2: Lớp tính giá cơ bản (Concrete Component)

Java

```
class BaseTripPricing implements ITripPricing {  
    private double distancePrice;  
  
    public BaseTripPricing(double distancePrice) {  
        this.distancePrice = distancePrice;  
    }  
  
    @Override  
    public double calculatePrice() {  
        return this.distancePrice; // Trả về giá cơ bản dựa trên quãng đường
```

```
}  
}
```

### **Bước 3: Lớp Decorator trừu tượng cho các loại phụ phí**

Java

```
abstract class PricingDecorator implements ITripPricing {  
    protected ITripPricing tripPricing;
```

```
  
    public PricingDecorator(ITripPricing pricing) {  
        this.tripPricing = pricing;  
    }  

```

```
@Override
```

```
    public double calculatePrice() {  
        return tripPricing.calculatePrice();  
    }  
}
```

### **Bước 4: Các lớp phụ phí cụ thể (Concrete Decorators)**

Java

```
// Phụ phí giờ cao điểm (Cộng thêm %)
```

```
class PeakHourSurcharge extends PricingDecorator {  
    public PeakHourSurcharge(ITripPricing pricing) {  
        super(pricing);  
    }  

```

```
@Override
```

```
public double calculatePrice() {  
    return super.calculatePrice() * 1.2; // Tăng 20% giá  
}  
}
```

// Phụ phí sân bay (Cộng tiền mặt)

```
class AirportSurcharge extends PricingDecorator {  
    public AirportSurcharge(ITripPricing pricing) {  
        super(pricing);  
    }  
}
```

@Override

```
public double calculatePrice() {  
    return super.calculatePrice() + 30000; // Cộng thêm 30,000đ  
}  
}
```

// Mã giảm giá (Trừ tiền)

```
class DiscountDecorator extends PricingDecorator {  
    private double discountAmount;  
  
    public DiscountDecorator(ITripPricing pricing, double amount) {  
        super(pricing);  
        this.discountAmount = amount;  
    }  
}
```



```
@Override  
  
public double calculatePrice() {  
    return Math.max(0, super.calculatePrice() - discountAmount);  
}  
}
```

### **Bước 5: Minh họa cách sử dụng tại runtime**

Java

```
public class PricingSystem {  
  
    public static void main(String[] args) {  
        // 1. Khởi tạo giá cơ bản  
        ITripPricing myTrip = new BaseTripPricing(50000);  
  
        // 2. Nếu là giờ cao điểm, áp dụng phụ phí  
        if (isPeakHour) {  
            myTrip = new PeakHourSurcharge(myTrip);  
        }  
  
        // 3. Nếu điểm đón/đến là sân bay  
        if (isAirport) {  
            myTrip = new AirportSurcharge(myTrip);  
        }  
  
        // 4. Áp dụng mã giảm giá nếu có  
        myTrip = new DiscountDecorator(myTrip, 10000);  
    }  
}
```

```
// Kết quả cuối cùng  
double finalPrice = myTrip.calculatePrice();  
System.out.println("Tổng tiền chuyến đi: " + finalPrice);  
}  
}
```

### Câu 3

Hệ thống có nhiều cách điều phối tài xế:

- Gần nhất
- Điểm đánh giá cao nhất
- Tối ưu theo thời gian hoàn thành
- Cân bằng số cuộc giữa tài xế

Quản trị có thể đổi cách điều phối theo thời gian hoặc khu vực.

#### Yêu cầu:

1. Đề xuất Design Pattern phù hợp.
2. Giải thích khả năng thay đổi thuật toán runtime.
3. Pseudo-code minh họa.

### Giải

#### 1. Đề xuất Design Pattern: Strategy Pattern (Mẫu thiết kế Chiến lược)

Để giải quyết bài toán thay đổi thuật toán điều phối tài xế dựa trên cấu hình quản trị, **Strategy Pattern** là lựa chọn tối ưu. Mẫu thiết kế này cho phép định nghĩa một tập hợp các thuật toán, đóng gói từng thuật toán lại và khiến chúng có thể thay đổi linh hoạt cho nhau.

## 2. Giải thích khả năng thay đổi thuật toán tại thời điểm chạy (Runtime)

Khả năng thay đổi linh hoạt của Strategy Pattern dựa trên các yếu tố kỹ thuật sau:

- **Tính đa hình (Polymorphism):** Hệ thống điều phối (Context) chỉ giao tiếp với các chiến lược thông qua một Interface chung. Tại thời điểm chạy, đối tượng cụ thể được gán cho Interface này có thể là bất kỳ thuật toán nào (Gần nhất, Đánh giá cao, Cân bằng cước...).
- **Tách biệt logic xử lý:** Logic của từng thuật toán điều phối được cô lập trong các lớp riêng biệt. Điều này giúp việc thay đổi thuật toán này không ảnh hưởng đến thuật toán kia và không làm gián đoạn luồng xử lý chính của hệ thống.
- **Cơ chế Dependency Injection:** Thông qua các phương thức như `setStrategy()` hoặc các bộ quản lý cấu hình (Configuration Manager), quản trị viên có thể thay đổi tham chiếu của chiến lược điều phối ngay lập tức mà không cần khởi động lại hệ thống hoặc biên dịch lại mã nguồn.
- **Khả năng mở rộng (Scalability):** Khi cần thêm phương thức điều phối mới (ví dụ: điều phối bằng AI), lập trình viên chỉ cần tạo thêm một lớp cụ thể thực thi Interface mà không cần can thiệp vào mã nguồn điều phối gốc, đảm bảo tính ổn định của hệ thống.

## 3. Pseudo-code minh họa (Java)

### Bước 1: Định nghĩa Interface cho chiến lược điều phối

```
interface IDispatchStrategy {  
    Driver findDriver(TripRequest request, List<Driver> availableDrivers);  
}
```

### Bước 2: Triển khai các thuật toán điều phối cụ thể

```
// Điều phối tài xế gần nhất
```

```
class NearestDriverStrategy implements IDispatchStrategy {
```

```

@Override

public Driver findDriver(TripRequest request, List<Driver> drivers) {

    // Logic tính toán khoảng cách GPS và trả về tài xế gần nhất

    return drivers.stream()

        .min(Comparator.comparing(d ->
calculateDistance(request.getPickupLocation(), d.getLocation()))

        .orElse(null);

    }

}

```

// Điều phối dựa trên điểm đánh giá cao nhất

```

class HighestRatingStrategy implements IDispatchStrategy {

    @Override

    public Driver findDriver(TripRequest request, List<Driver> drivers) {

        return drivers.stream()

            .max(Comparator.comparing(Driver::getRating))

            .orElse(null);

    }

}

```

// Điều phối cân bằng số cuộc (Load Balancing)

```

class LoadBalancingStrategy implements IDispatchStrategy {

    @Override

    public Driver findDriver(TripRequest request, List<Driver> drivers) {

        // Ưu tiên tài xế có ít cuộc xe nhất trong ngày
    }

}

```

```

        return drivers.stream()

            .min(Comparator.comparing(Driver::getTodayTripCount))

            .orElse(null);
    }
}

```

### **Bước 3: Lớp điều phối đóng vai trò Context**

```

class DriverDispatcher {

    private IDispatchStrategy strategy;

    // Thay đổi thuật toán tại thời điểm chạy (Runtime)
    public void setStrategy(IDispatchStrategy strategy) {

        this.strategy = strategy;
    }

    public void executeDispatch(TripRequest request, List<Driver>
availableDrivers) {

        Driver selectedDriver = strategy.findDriver(request, availableDrivers);

        if (selectedDriver != null) {

            selectedDriver.notifyNewTrip(request);

            System.out.println("Đã điều phối tài xế: " +
selectedDriver.getName());

        }

    }
}

```

### **Bước 4: Sử dụng trong hệ thống thực tế**

Java

```
public class Main {  
    public static void main(String[] args) {  
        Dispatcher dispatcher = new Dispatcher();  
        List<Driver> drivers = getOnlineDrivers();  
  
        // Giả sử quản trị viên đổi cấu hình sang "Ưu tiên đánh giá" cho khu vực  
        Quận 1  
        dispatcher.setStrategy(new HighestRatingStrategy());  
        dispatcher.executeDispatch(new TripRequest, drivers);  
  
        // Sau 22h, đổi sang chiến lược "Cân bằng số cuộc"  
        dispatcher.setStrategy(new LoadBalancingStrategy());  
        dispatcher.executeDispatch(anotherTripRequest, drivers);  
    }  
}
```

## Câu 4

Khi trạng thái chuyến đi thay đổi (accepted, arrived, started, completed):

- App khách cần cập nhật
- App tài xế cần cập nhật
- Hệ thống gửi thông báo
- Dashboard cập nhật số liệu

Các thành phần này hoạt động độc lập.

**Yêu cầu:**

1. Đề xuất Design Pattern phù hợp.
2. Giải thích vì sao phù hợp với hệ thống realtime.
3. Pseudo-code minh họa cơ chế thông báo.

## **Giải**

### **1. Đề xuất Design Pattern: Observer Pattern (Mẫu thiết kế Người quan sát)**

Để giải quyết bài toán cập nhật thông tin đồng thời cho nhiều thành phần độc lập khi trạng thái chuyển đi thay đổi, Observer Pattern là lựa chọn phù hợp nhất. Mẫu thiết kế này định nghĩa một mối quan hệ phụ thuộc một - nhiều giữa các đối tượng, sao cho khi một đối tượng thay đổi trạng thái, tất cả các đối tượng phụ thuộc của nó đều được thông báo và cập nhật tự động.

### **2. Giải thích lý do phù hợp với hệ thống Real-time**

Observer Pattern mang lại những lợi điểm then chốt cho một hệ thống yêu cầu tính tức thời như ứng dụng đặt xe:

- Tính lỏng lẻo (Loose Coupling): Đối tượng quản lý chuyển đi (Subject) không cần biết chi tiết về các thành phần nhận tin (App khách, Dashboard, ...). Nó chỉ duy trì một danh sách các "người quan sát". Điều này cho phép thêm hoặc bớt các thành phần nhận thông báo tại runtime mà không ảnh hưởng đến logic cốt lõi của chuyển đi.
- Phản hồi tức thì (Immediate Updates): Ngay khi tài xế nhấn "Chấp nhận" hoặc "Hoàn thành", trạng thái mới sẽ được đẩy (push) ngay lập tức tới các bên liên quan. Điều này đảm bảo tính nhất quán dữ liệu giữa App tài xế và App khách hàng trong thời gian thực.
- Khả năng mở rộng (Scalability): Trong tương lai, nếu hệ thống cần thêm các tính năng mới như "Gửi hóa đơn qua Email" hoặc "Ghi Log vào Big Data" khi chuyển đi kết thúc, lập trình viên chỉ cần tạo thêm

một Observer mới và đăng ký vào hệ thống mà không phải sửa đổi mã nguồn của lớp Trip.

- Tách biệt trách nhiệm (Separation of Concerns): Logic xử lý trạng thái chuyến đi và logic gửi thông báo (Push Notification, Socket.io, cập nhật database) được tách biệt hoàn toàn, giúp hệ thống dễ bảo trì và kiểm thử.

### **3. Pseudo-code minh họa cơ chế thông báo (Java)**

#### **Bước 1: Định nghĩa Interface cho người quan sát (Observer)**

```
interface ITripObserver {  
    void onTripStatusChanged(Trip trip);  
}
```

#### **Bước 2: Lớp Trip đóng vai trò là Subject (Đối tượng được quan sát)**

```
class Trip {  
    private List<ITripObserver> observers = new ArrayList<>();  
    private String status; // accepted, arrived, started, completed  
    private String tripId;  
  
    public void addObserver(ITripObserver observer) {  
        observers.add(observer);  
    }  
  
    public void removeObserver(ITripObserver observer) {  
        observers.remove(observer);  
    }  
}
```



```
// Khi trạng thái thay đổi, thông báo cho tất cả các bên
```

```
public void setStatus(String newStatus) {  
    this.status = newStatus;  
    notifyAllObservers();  
}
```

```
private void notifyAllObservers() {  
    for (ITripObserver observer : observers) {  
        observer.onTripStatusChanged(this);  
    }  
}
```

```
// Getters...
```

```
}
```

### **Bước 3: Triển khai các thành phần nhận thông báo (Concrete Observers)**

```
// Cập nhật cho ứng dụng Khách hàng qua WebSocket/Firebase
```

```
class CustomerAppNotifier implements ITripObserver {
```

```
    @Override
```

```
    public void onTripStatusChanged(Trip trip) {
```

```
        System.out.println("[Firebase] Gửi thông báo tới Khách hàng: Chuyển đi  
" + trip.getTripId() + " hiện là " + trip.getStatus());
```

```
    }
```

```
}
```

```
// Cập nhật số liệu trên Dashboard quản trị
```

```

class AdminDashboardUpdater implements ITripObserver {

    @Override

    public void onTripStatusChanged(Trip trip) {

        if (trip.getStatus().equals("completed")) {

            System.out.println("[Database] Cập nhật tổng doanh thu và số
chuyến đi thành công trên Dashboard.");

        }

    }

}

// Hệ thống gửi SMS/Email thông báo

class NotificationService implements ITripObserver {

    @Override

    public void onTripStatusChanged(Trip trip) {

        System.out.println("[SMS] Đang gửi thông báo trạng thái mới cho các
bên liên quan...");

    }

}

```

#### **Bước 4: Vận hành cơ chế**

```

public class TripManagementSystem {

    public static void main(String[] args) {

        Trip trip = new Trip("TRIP123");

        // Đăng ký các thành phần muốn theo dõi chuyến đi

        trip.addObserver(new CustomerAppNotifier());

        trip.addObserver(new AdminDashboardUpdater());
    }

}

```

```
trip.addObserver(new NotificationService());

// Giả lập sự kiện tài xế cập nhật trạng thái
System.out.println("--- Tài xế đã đến điểm đón ---");
trip.setStatus("arrived");

System.out.println("--- Tài xế kết thúc chuyến đi ---");
trip.setStatus("completed");
}
}
```

### Câu 5

Khách có thể gửi khiếu nại hoàn tiền. Quy trình xử lý gồm nhiều bước:

- Kiểm tra hợp lệ
- Kiểm tra chính sách
- Kiểm tra gian lận
- Phê duyệt hoặc từ chối

Quy trình có thể thay đổi hoặc thêm bước mới.

### Yêu cầu:

1. Đề xuất Design Pattern phù hợp.
2. Giải thích khả năng mở rộng quy trình.
3. Pseudo-code minh họa

### Giải

#### 1. Đề xuất Design Pattern: Chain of Responsibility (Mẫu thiết kế Chuỗi trách nhiệm)

Để giải quyết bài toán quy trình xử lý gồm nhiều bước kiểm tra tuần tự và có khả năng thay đổi, Chain of Responsibility là lựa chọn tối ưu. Mẫu thiết kế này cho phép chuyển các yêu cầu dọc theo một chuỗi các trình xử lý (handlers). Khi nhận được một yêu cầu, mỗi trình xử lý sẽ quyết định xử lý yêu cầu đó hoặc chuyển nó cho trình xử lý tiếp theo trong chuỗi.

## 2. Giải thích khả năng mở rộng quy trình

Mẫu thiết kế Chain of Responsibility mang lại khả năng mở rộng cực kỳ linh hoạt cho hệ thống:

- Tách biệt các bước xử lý (Decoupling): Mỗi bước trong quy trình (Kiểm tra hợp lệ, Kiểm tra gian lận,...) được đóng gói trong một lớp riêng biệt. Đối tượng gửi yêu cầu hoàn toàn không cần biết bước nào đang xử lý nó hoặc cấu trúc của chuỗi xử lý ra sao.
- Nguyên tắc Đơn nhiệm (Single Responsibility Principle): Mỗi lớp chỉ tập trung vào một logic kiểm tra duy nhất. Điều này giúp mã nguồn dễ đọc, dễ bảo trì và kiểm thử độc lập.
- Thay đổi quy trình linh hoạt (Dynamic Configuration): Có thể thay đổi thứ tự các bước, thêm bước mới (ví dụ: "Xác nhận của quản lý") hoặc loại bỏ một bước chỉ bằng cách thay đổi việc gán tham chiếu nextHandler trong chuỗi mà không cần sửa đổi mã nguồn của các lớp hiện có (tuân thủ nguyên tắc Open/Closed).
- Cơ chế ngắt sớm: Nếu một bước kiểm tra thất bại (ví dụ: phát hiện gian lận), trình xử lý có thể dừng chuỗi ngay lập tức và trả về kết quả từ chối mà không cần chạy qua các bước phía sau, giúp tối ưu hiệu năng.

## 3. Pseudo-code minh họa (Java)

### Bước 1: Lớp dữ liệu yêu cầu hoàn tiền

```
class RefundRequest {
```

```
private String tripld;  
private double amount;  
private String reason;  
// Getters and Setters...  
}
```

## **Bước 2: Lớp trừu tượng Handler**

```
abstract class RefundHandler {  
    protected RefundHandler nextHandler;  
  
    public void setNext(RefundHandler nextHandler) {  
        this.nextHandler = nextHandler;  
    }  
  
    public abstract void process(RefundRequest request);  
  
    protected void passToNext(RefundRequest request) {  
        if (nextHandler != null) {  
            nextHandler.process(request);  
        } else {  
            System.out.println("Quy trình hoàn tất: Phê duyệt hoàn tiền.");  
        }  
    }  
}
```

## **Bước 3: Các trình xử lý cụ thể**

```
// 1. Kiểm tra hợp lệ (Ví dụ: Chuyển đi có tồn tại không?)
```

```
class ValidationHandler extends RefundHandler {  
    public void process(RefundRequest request) {  
        if (isValid(request)) {  
            System.out.println("Bước 1: Kiểm tra hợp lệ thành công.");  
            passToNext(request);  
        } else {  
            System.out.println("Từ chối: Yêu cầu không hợp lệ.");  
        }  
    }  
}
```

// 2. Kiểm tra chính sách (Ví dụ: Thời hạn khiếu nại còn không?)

```
class PolicyHandler extends RefundHandler {  
    public void process(RefundRequest request) {  
        if (isMeetingPolicy(request)) {  
            System.out.println("Bước 2: Kiểm tra chính sách thành công.");  
            passToNext(request);  
        } else {  
            System.out.println("Từ chối: Vi phạm chính sách hoàn tiền.");  
        }  
    }  
}
```

// 3. Kiểm tra gian lận (Fraud Check)

```
class FraudCheckHandler extends RefundHandler {
```

```

public void process(RefundRequest request) {
    if (!isFraud(request)) {
        System.out.println("Bước 3: Không phát hiện gian lận.");
        passToNext(request);
    } else {
        System.out.println("Từ chối: Phát hiện hành vi gian lận.");
    }
}
}

```

#### **Bước 4: Thiết lập và thực thi chuỗi**

```

public class RefundSystem {
    public static void main(String[] args) {
        // Khởi tạo các bước
        RefundHandler step1 = new ValidationHandler();
        RefundHandler step2 = new PolicyHandler();
        RefundHandler step3 = new FraudCheckHandler();

        // Kết nối chuỗi: Step 1 -> Step 2 -> Step 3
        step1.setNext(step2);
        step2.setNext(step3);

        // Bắt đầu xử lý yêu cầu
        RefundRequest myRequest = new RefundRequest("TRIP789", 50000,
            "Tài xế không đến");
        step1.process(myRequest);
    }
}

```

}

}